Taller Deep Learning con PyTorch 2026

MC. Xavier Sierra-Canto

Instituto Tecnológico de Mérida

Noviembre de 2026

Objetivo del taller

Al terminar, serás capaz de...

- Explicar cómo *aprende* una red neuronal.
- Entrenar una red en PyTorch siguiendo siempre la misma metodología.
- Construir una CNN que clasifique imágenes médicas (PneumoniaMNIST).

Objetivo del taller

Al terminar, serás capaz de...

- Explicar cómo *aprende* una red neuronal.
- Entrenar una red en PyTorch siguiendo siempre la misma metodología.
- Construir una CNN que clasifique imágenes médicas (PneumoniaMNIST).

Hilo conductor

Todo el taller gira en torno a **una metodología de 5 pasos**. Cambian los datos y el modelo; la metodología no.

La metodología de entrenamiento



La metodología de entrenamiento



- Sesión 1.1: entendemos **por qué** funciona (teoría).

La metodología de entrenamiento



- Sesión 1.1: entendemos **por qué** funciona (teoría).
- Sesión 1.2: la aplicamos a puntos en 2D (*moons*).

La metodología de entrenamiento



- Sesión 1.1: entendemos **por qué** funciona (teoría).
- Sesión 1.2: la aplicamos a puntos en 2D (*moons*).
- Sesión 2: la aplicamos a imágenes (CNN y PneumoniaMNIST).

Plan de sesiones

Sesión	Tema	Pregunta guía
1.1	Introducción a ML y DL	¿Qué problema resolvemos?
1.2	Redes neuronales y backpropagation	¿Cómo aprende una red?
1.3	ANN con PyTorch	¿Cómo lo programo?
2	CNN con PyTorch	¿Y si la entrada es una imagen?

¿Dónde estamos?

1. Introducción al aprendizaje automático
2. Redes neuronales y backpropagation
3. Construyendo redes con PyTorch
   - Herramientas básicas
   - Descenso de gradiente paso a paso
   - Nuestra primera red: moons
4. Introducción a CNN con PyTorch
   - Calculando dimensiones
   - Las capas de una CNN con Sequential
   - Proyecto: PneumoniaMNIST
5. Cierre

Machine Learning

- Disciplina de la Inteligencia Artificial en la que los algoritmos **identifican patrones** en grandes cantidades de datos y **realizan predicciones**.
- En lugar de programar reglas, *mostramos ejemplos*.

Machine Learning

- Disciplina de la Inteligencia Artificial en la que los algoritmos **identifican patrones** en grandes cantidades de datos y **realizan predicciones**.
- En lugar de programar reglas, *mostramos ejemplos*.

Tipos de aprendizaje

Supervisado	Datos con etiqueta (*este es el foco del taller*).
No supervisado	Datos sin etiqueta; se buscan estructuras.
Por refuerzo	Un agente aprende por recompensas.

Deep Learning

- Modelo de aprendizaje basado en **redes neuronales artificiales**.
- Tienen muchas capas, de ahí el término "profundo".
- Cada capa transforma los datos en representaciones cada vez más abstractas.

$$\text{ML} \supset \text{Redes neuronales} \supset \text{Deep Learning}$$

Ejemplos de aplicación

- MusicVAE: `https://magenta.tensorflow.org/music-vae`
- MelNet: `https://sjvasquez.github.io/blog/melnet/`
- Video: `https://www.youtube.com/watch?v=qv6UVOQ0F44`

¿Dónde estamos?

1. Introducción al aprendizaje automático
2. Redes neuronales y backpropagation
3. Construyendo redes con PyTorch
   - Herramientas básicas
   - Descenso de gradiente paso a paso
   - Nuestra primera red: moons
4. Introducción a CNN con PyTorch
   - Calculando dimensiones
   - Las capas de una CNN con Sequential
   - Proyecto: PneumoniaMNIST
5. Cierre

Un problema nuestro



Pregunta

¿Podemos predecir el **sexo** de una persona a partir de su **estatura** y su **peso**?

Encuesta **anónima y voluntaria** a todo el grupo. A cada persona le preguntamos:

1. Estatura (cm)
2. Peso (kg)
3. Sexo (0 = mujer, 1 = hombre)

Los datos del grupo

#	Estatura (cm)	Peso (kg)	Sexo (0/1)
1			
2			
3			
4			
5			
⋮			

Entradas: estatura y peso $\rightarrow$ Salida: sexo (clasificación binaria)

Visualizando los datos

[falta imagenes/datos_grupo.png]

Cada punto es una persona. ¿Dónde dibujarían una **recta** que separe los dos grupos?

Planteando el problema

Una **sola neurona** con dos entradas:

$$z = w_1 \,\text{estatura} + w_2 \,\text{peso} + b, \qquad \hat{y} = \sigma(z)$$

$\hat{y}$ es la probabilidad de la clase 1; decidimos clase 1 si $\hat{y} > 0{,}5$. La frontera $z = 0$ es una **recta**.

Paso de la metodología	**En este problema**
1. Datos	la tabla del grupo (matriz $n \times 2$ y vector de n etiquetas)
2. Modelo	una neurona con sigmoide
3. Pérdida	entropía cruzada binaria
4. Optimizador	descenso de gradiente, *a mano*
5. Bucle	forward $\rightarrow$ gradientes $\rightarrow$ actualizar

Planteando el problema

Una **sola neurona** con dos entradas:

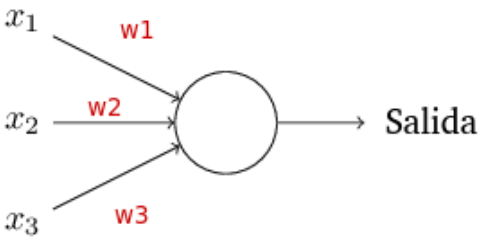
$$z = w_1 \,\text{estatura} + w_2 \,\text{peso} + b, \qquad \hat{y} = \sigma(z)$$

$\hat{y}$ es la probabilidad de la clase 1; decidimos clase 1 si $\hat{y} > 0{,}5$. La frontera $z = 0$ es una **recta**.

Paso de la metodología	**En este problema**
1. Datos	la tabla del grupo (matriz $n \times 2$ y vector de n etiquetas)
2. Modelo	una neurona con sigmoide
3. Pérdida	entropía cruzada binaria
4. Optimizador	descenso de gradiente, *a mano*
5. Bucle	forward $\rightarrow$ gradientes $\rightarrow$ actualizar

Para discutir: ¿una sola neurona será suficiente? ¿Qué pasa con las personas que quedan cerca de la recta?

La neurona artificial



Inspirada en la neurona biológica. Calcula una suma ponderada de sus entradas más un sesgo b:

$$z = \sum_i w_i x_i + b$$

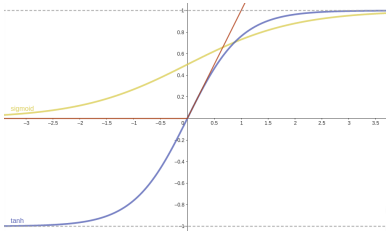
Perceptrón (umbral):

$$\text{Salida} = \begin{cases} 1, & \text{si } z > 0 \\ 0, & \text{si } z \leq 0 \end{cases}$$

Funciones de activación

Un umbral duro no permite aprender con gradientes; usamos funciones suaves:

- Sigmoide: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$
- Tanh: $\tanh(z) = \dfrac{\sinh(z)}{\cosh(z)}$
- ReLU: $\max(0, z)$



Paso 3 de la metodología: la función de pérdida



Mide qué tan equivocadas están las predicciones (también: función de costo o error).

- Regresión, error cuadrático medio (MSE):

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Clasificación, entropía cruzada (binaria):

$$-\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

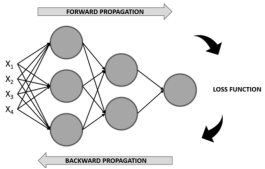
Pasos 4 y 5: cómo aprende la red



1. Se calculan las salidas (*forward*).
2. Se mide el error con la función de pérdida.
3. Con la regla de la cadena se obtiene el gradiente de la pérdida respecto a cada peso (*backpropagation*).
4. Se actualizan los pesos **en dirección contraria al gradiente**:

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

donde α es el *learning rate*.



Una neurona desde cero (1): datos y parámetros




```python
import numpy as np

# X: (n, 2) = estatura, peso       y: (n,) con 0 / 1
media = X.mean(axis=0)
desv = X.std(axis=0)
X_norm = (X - media) / desv          # normalizar

w = np.zeros(2)                      # un peso por entrada
b = 0.0
lr = 0.1

def sigmoide(z):
    return 1 / (1 + np.exp(-z))
```

La estatura ronda 170 y el peso 70: las escalas son parecidas, pero ninguna está centrada en 0. Lo probaremos en el notebook.

El gradiente (1): la regla de la cadena



Para una persona i, el cálculo es una cadena de tres pasos:

$$z = \mathrm{w} \cdot \mathrm{x} + b \longrightarrow \hat{y} = \sigma(z) \longrightarrow \ell = -\big[y \log \hat{y} + (1-y)\log(1-\hat{y})\big]$$

Queremos saber cómo cambia el error ℓ al mover cada peso. Se multiplican las derivadas de cada eslabón:

$$\frac{\partial \ell}{\partial w_j} = \underbrace{\frac{\partial \ell}{\partial \hat{y}}}_{\text{pérdida}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{\text{sigmoide}} \cdot \underbrace{\frac{\partial z}{\partial w_j}}_{\text{suma ponderada}}$$

El gradiente (2): cada factor por separado

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}\,(1-\hat{y})}$$

$$\frac{\partial \hat{y}}{\partial z} = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\big(1-\sigma(z)\big) = \hat{y}\,(1-\hat{y})$$

$$\frac{\partial z}{\partial w_j} = x_j \qquad \frac{\partial z}{\partial b} = 1$$

El gradiente (2): cada factor por separado

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}\,(1-\hat{y})}$$

$$\frac{\partial \hat{y}}{\partial z} = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\big(1-\sigma(z)\big) = \hat{y}\,(1-\hat{y})$$

$$\frac{\partial z}{\partial w_j} = x_j \qquad\qquad \frac{\partial z}{\partial b} = 1$$

Al multiplicar los dos primeros, el denominador se cancela:

$$\frac{\partial \ell}{\partial z} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y$$

El gradiente (3): de una persona a todo el grupo

Para una persona: $\dfrac{\partial \ell}{\partial w_j} = (\hat{y} - y)\, x_j$ y $\dfrac{\partial \ell}{\partial b} = \hat{y} - y$.

Promediando las n personas:

$$\frac{\partial L}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)\, x_{ij} \implies \frac{\partial L}{\partial \mathrm{w}} = \frac{1}{n} X^T (\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

El gradiente (3): de una persona a todo el grupo

Para una persona: $\dfrac{\partial \ell}{\partial w_j} = (\hat{y} - y)\, x_j$ y $\dfrac{\partial \ell}{\partial b} = \hat{y} - y$.

Promediando las n personas:

$$\frac{\partial L}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\, x_{ij} \implies \frac{\partial L}{\partial \mathrm{w}} = \frac{1}{n} X^T(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

Y en el código

```
dz = y_hat - y
dw = X_norm.T @ dz / len(y)
db = dz.mean()
```

El gradiente (4): ¿por qué esta pareja funciona bien?

- $\hat{y} - y$ es simplemente el **error de la predicción**: positivo si la neurona estimó de más, negativo si estimó de menos.
- El peso w_j se corrige en proporción a ese error y a la entrada x_j que participó.

El gradiente (4): ¿por qué esta pareja funciona bien?

- $\hat{y} - y$ es simplemente el **error de la predicción**: positivo si la neurona estimó de más, negativo si estimó de menos.
- El peso w_j se corrige en proporción a ese error y a la entrada x_j que participó.
- Con sigmoide y *error cuadrático*, aparecería un factor extra $\hat{y}(1 - \hat{y})$ que casi se anula cuando la sigmoide está saturada, y el aprendizaje se frena aun con predicciones muy malas.
- La **entropía cruzada** cancela ese factor: por eso es la pérdida natural para clasificación.

Actualización de los pesos

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w} \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

El gradiente apunta hacia donde la pérdida **crece**; por eso nos movemos en sentido contrario.

Actualización de los pesos

$$\mathrm{w} \leftarrow \mathrm{w} - \alpha \frac{\partial L}{\partial \mathrm{w}} \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

El gradiente apunta hacia donde la pérdida **crece**; por eso nos movemos en sentido contrario.

Un paso a mano (una persona con $\mathrm{x} = (1, 2)$, $y = 1$; $\mathrm{w} = 0$, $b = 0$, $\alpha = 0{,}1$):

$$\begin{array}{ll} z = 0 \;\Rightarrow\; \hat{y} = 0{,}5 & \hat{y} - y = -0{,}5 \\ \partial L/\partial \mathrm{w} = -0{,}5\,(1, 2) = (-0{,}5,\ -1) & \partial L/\partial b = -0{,}5 \\ \mathrm{w} = (0{,}05,\ 0{,}1) & b = 0{,}05 \end{array}$$

Actualización de los pesos

$$\mathrm{w} \leftarrow \mathrm{w} - \alpha \frac{\partial L}{\partial \mathrm{w}} \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

El gradiente apunta hacia donde la pérdida **crece**; por eso nos movemos en sentido contrario.

Un paso a mano (una persona con $\mathrm{x} = (1, 2)$, $y = 1$; $\mathrm{w} = 0$, $b = 0$, $\alpha = 0{,}1$):

$$\begin{array}{ll} z = 0 \;\Rightarrow\; \hat{y} = 0{,}5 & \hat{y} - y = -0{,}5 \\ \partial L/\partial \mathrm{w} = -0{,}5\,(1, 2) = (-0{,}5,\ -1) & \partial L/\partial b = -0{,}5 \\ \mathrm{w} = (0{,}05,\ 0{,}1) & b = 0{,}05 \end{array}$$

Ahora $z = 0{,}3$ y $\hat{y} \approx 0{,}57$: la estimación se acercó a $y = 1$.

Una neurona desde cero (3): el bucle




```
for epoca in range(500):
    z = X_norm @ w + b                          # forward
    y_hat = sigmoide(z)
    loss = -np.mean(y*np.log(y_hat) + (1 - y)*np.log(1 - y_hat))

    dz = y_hat - y                              # gradientes
    dw = X_norm.T @ dz / len(y)
    db = dz.mean()

    w = w - lr * dw                             # actualizar
    b = b - lr * db
```

Viendo cómo aprende la neurona

[falta imagenes/recta_epocas.png]

La recta en las épocas 1, 10, 50 y 500: empieza equivocada y se ajusta a los datos.

De NumPy a PyTorch

Lo que escribimos a mano tiene su equivalente en PyTorch:

A mano (NumPy)	PyTorch
`w, b`	`nn.Linear(2, 1)`
`sigmoide(z)`	`nn.Sigmoid()`
`loss = ...`	`nn.BCELoss()`
`dz, dw, db`	`loss.backward()`
`w = w - lr * dw`	`optimizador.step()`

La metodología es la misma; PyTorch automatiza los pasos más tediosos.

Ventajas y desventajas de las redes neuronales

Ventajas

- Buen desempeño con grandes volúmenes de datos.
- Resuelven problemas complejos mejor que otros algoritmos.
- El hardware actual potencia sus capacidades.
- Área activa de investigación.

Desventajas

- Difícil entender la convergencia y las causas de los errores.
- Requieren muchos datos.
- Entrenamientos largos.
- Mucha potencia de cómputo.

Resumen

- Neurona: $z = \sum w_i x_i + b$ seguida de una activación.
- La pérdida mide el error; el gradiente indica cómo corregirlo.
- Aprender = repetir: *forward* $\rightarrow$ *pérdida* $\rightarrow$ *backward* $\rightarrow$ *actualizar*.

Siguiente pregunta: ¿cómo calcula PyTorch esos gradientes por nosotros?

¿Dónde estamos?

1. Introducción al aprendizaje automático
2. Redes neuronales y backpropagation
3. Construyendo redes con PyTorch
   - Herramientas básicas
   - Descenso de gradiente paso a paso
   - Nuestra primera red: moons
4. Introducción a CNN con PyTorch
   - Calculando dimensiones
   - Las capas de una CNN con Sequential
   - Proyecto: PneumoniaMNIST
5. Cierre

Introducción a PyTorch

- Biblioteca de Python para cómputo numérico de alto rendimiento.
- Similar a NumPy, pero **puede ejecutarse en GPU**.
- Flexible y rápida para implementar redes profundas.

Tensores

- Arreglos multidimensionales (escalares, vectores, matrices, ...).
- Pueden moverse a GPU con `.to(device)`.

1. FloatTensor: 32-bit float
2. DoubleTensor: 64-bit float
3. HalfTensor: 16-bit float
4. IntTensor: 32-bit int
5. LongTensor: 64-bit int

Ejemplo: crear tensores

```
import torch
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print("Usando:", device)

tensor_1 = torch.tensor([0.1, 1, 0.9, 0.7, 0.3])
tensor_2 = torch.tensor([[0, 0.2, 0.4, 0.6],
                         [1, 0.8, 0.6, 0.4]]).to(device)
print(tensor_1.shape)   # torch.Size([5])
print(tensor_2.shape)   # torch.Size([2, 4])
```

Diferenciación automática (Autograd)

- Calcula derivadas en un punto de forma automática.
- Es lo que hace posible el *backpropagation*: no derivamos a mano.
- PyTorch registra las operaciones hacia adelante (grafo computacional) y las recorre hacia atrás con `.backward()`.

Autograd en acción

```
import torch
a = torch.ones((2, 2), requires_grad=True)
b = a + 5
c = b.mean()

print('b =', b)
print('c =', c)

c.backward()       # backpropagation
print(a.grad)      # dc/da = 1/4 en cada elemento
```

Optimizadores

SGD	Actualiza con el gradiente de un lote (*batch*) de ejemplos.
AdaGrad	Learning rate distinto para cada peso.
Adam	Además usa el momento de los gradientes anteriores; buena opción por defecto.
L-BFGS	Método de segundo orden aproximado.

Optimizadores en PyTorch

```
from torch import optim

adam = optim.Adam(model.parameters(), lr=learning_rate)
sgd  = optim.SGD(model.parameters(),  lr=learning_rate)
```

Un ejemplo mínimo

Función objetivo y su gradiente:

$$f(x) = (x-3)^2 + 5 \qquad \nabla_x f = 2(x-3)$$

Regla de actualización (`optimizador.step()`, con $\eta = 0{,}1$):

$$x_{nueva} = x_{actual} - \eta \nabla_x f(x_{actual})$$

Un ejemplo mínimo

Función objetivo y su gradiente:

$$f(x) = (x-3)^2 + 5 \qquad \nabla_x f = 2(x-3)$$

Regla de actualización (`optimizador.step()`, con $\eta = 0{,}1$):

$$x_{nueva} = x_{actual} - \eta \nabla_x f(x_{actual})$$

Partiendo de $x_0 = 0$:

iteración	0	1	2	3	4
x	0	0.60	1.08	1.46	1.77
$x-3$	-3	$-2{,}4$	$-1{,}92$	$-1{,}54$	$-1{,}23$

La distancia al mínimo ($x = 3$) se multiplica por 0,8 en cada paso.

Descenso de gradiente en PyTorch

```python
import torch
import torch.optim as optim

x = torch.tensor([0.0], requires_grad=True)
optimizador = optim.SGD([x], lr=0.1)

for i in range(30):
    optimizador.zero_grad()    # limpiar gradientes anteriores
    y = (x - 3)**2 + 5         # forward
    y.backward()               # backward
    optimizador.step()         # mover x hacia el minimo
    print(i, '|', x.item(), '|', y.item())

print(f"Minimo encontrado! x = {x.item():.4f}")
```

(Opcional) Graficar la función

```
import numpy as np
import matplotlib.pyplot as plt

a = np.linspace(0, 10, 100)
b = (a - 3)**2 + 5
plt.plot(a, b)
plt.show()
```

El módulo `torch.nn`

- Contiene lo necesario para crear y entrenar redes: capas, activaciones y funciones de pérdida.
- `nn.Sequential()` apila capas en orden.
- Activaciones: `nn.ReLU()`, `nn.LeakyReLU()`, `nn.Sigmoid()`, `nn.Tanh()`.
- Pérdidas: `nn.MSELoss()`, `nn.CrossEntropyLoss()`.

```
import torch.nn as nn
model = nn.Sequential(nn.Linear(input_units, hidden_units), \
                      nn.ReLU(), \
                      nn.Linear(hidden_units, output_units), \
                      nn.Sigmoid())
loss_funct = nn.MSELoss()
```

Problema: clasificar dos lunas



Dos clases de puntos en 2D que **no son separables con una línea recta**.

- Entrada: 2 valores (x_1, x_2).
- Salida: la clase (0 o 1).
- Arquitectura: $2 \to 16 \to 16 \to 2$ con ReLU.

Paso 1: Datos




```
import torch, torch.nn as nn, torch.optim as optim
from sklearn.datasets import make_moons

X_np, y_np = make_moons(n_samples=400, noise=0.15, random_state=42)
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.long)
```

Pasos 2, 3 y 4: Modelo, pérdida y optimizador




```
modelo_lunas = nn.Sequential(
    nn.Linear(2, 16), nn.ReLU(),
    nn.Linear(16, 16), nn.ReLU(),
    nn.Linear(16, 2)
)

criterio     = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo_lunas.parameters(), lr=0.01)
```

Paso 5: Bucle de entrenamiento




```
epocas = 500
for epoca in range(epocas):
    optimizador.zero_grad()                    # limpiar gradientes
    predicciones = modelo_lunas(X)             # forward
    loss = criterio(predicciones, y)           # perdida
    loss.backward()                            # backward
    optimizador.step()                         # actualizar pesos

    if (epoca + 1) % 100 == 0:
        print(f"Epoca {epoca+1} | Perdida: {loss.item():.4f}")
```

Actividad 1

Reto

Reproduce la red para *moons* y responde:

1. ¿Qué precisión obtiene sobre los datos de entrenamiento?
2. ¿Qué pasa si cambias `lr` a 0.001 o 0.5?
3. ¿Qué pasa si quitas las `ReLU`? ¿Por qué?
4. (Extra) Grafica la frontera de decisión.

Resumen de la sesión 3



- Autograd calcula los gradientes; el optimizador actualiza los pesos.
- `nn.Sequential` permite armar redes en pocas líneas.
- El bucle de entrenamiento siempre tiene el mismo orden: `zero_grad` → `forward` → `loss` → `backward` → `step`.

Siguiente pregunta: ¿qué cambia cuando la entrada es una imagen?

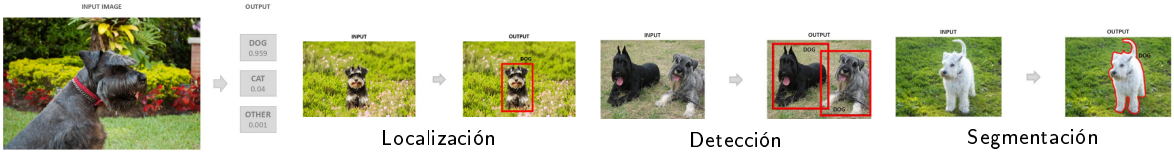
¿Dónde estamos?

1. Introducción al aprendizaje automático
2. Redes neuronales y backpropagation
3. Construyendo redes con PyTorch
   - Herramientas básicas
   - Descenso de gradiente paso a paso
   - Nuestra primera red: moons
4. Introducción a CNN con PyTorch
   - Calculando dimensiones
   - Las capas de una CNN con Sequential
   - Proyecto: PneumoniaMNIST
5. Cierre

¿Por qué CNN?

- Una imagen de 28×28 ya tiene 784 valores; una red densa ignora la **vecindad** entre píxeles.
- Las CNN trabajan con la imagen como matriz y reutilizan los mismos filtros en toda la imagen.
- Requieren poco preprocesamiento (normalizar valores y, a veces, reducir dimensiones).

Aplicaciones comunes



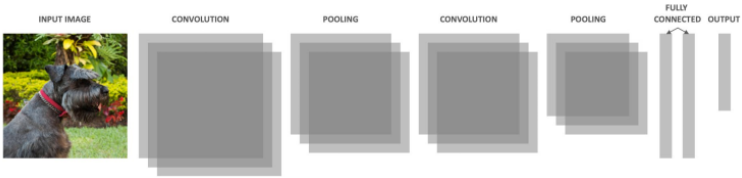
Clasificación Localización Detección Segmentación

Arquitectura de una CNN

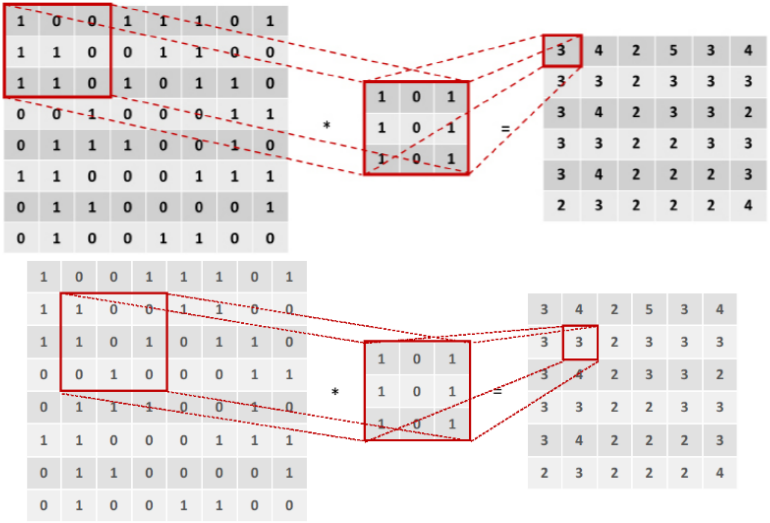
Capa convolucional Un filtro recorre la imagen y detecta patrones (bordes, texturas...).

Pooling Reduce el tamaño de la salida y elimina ruido.

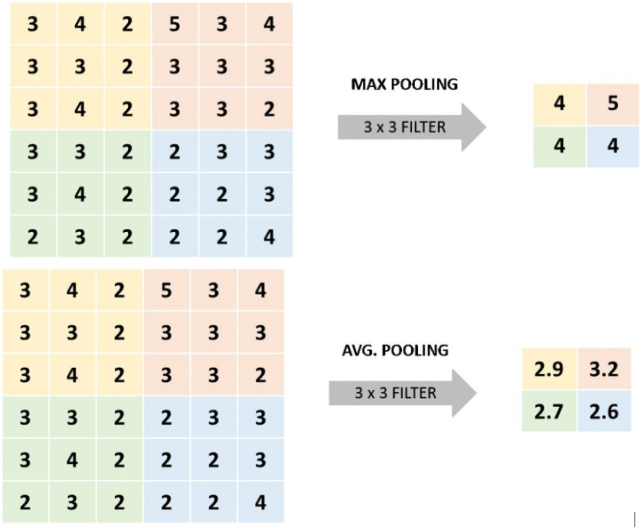
Completamente conectada Clasificador final: calcula la probabilidad de cada etiqueta.



Capa convolucional



Capa de pooling



Max pooling (izquierda) conserva el valor máximo; average pooling (derecha) el promedio.

Tamaño de salida de una convolución

$$n_{sal} = \left\lfloor \frac{n - k + 2p}{s} \right\rfloor + 1$$

n: tamaño de entrada, k: tamaño del filtro, p: padding, s: stride. Se aplica por separado al alto y al ancho.

Ejemplo con $n = 28$ y $k = 3$:

p	s	n_{sal}	Cálculo	
0	1	26	$\lfloor (28-3)/1 \rfloor + 1$	
1	1	28	$\lfloor (28-3+2)/1 \rfloor + 1$	(“same”)
0	2	13	$\lfloor (28-3)/2 \rfloor + 1$	
1	2	14	$\lfloor (28-3+2)/2 \rfloor + 1$	

- Con $k = 3$, $s = 1$ y $p = 1$ el tamaño **se conserva**.
- Los **canales** de salida son los que pidas en `out_channels`, sin importar k.

Parámetros de una capa convolucional

Cada filtro tiene forma `in_channels`$\times k \times k$, más un sesgo:

$$\text{parámetros} = \text{out_channels} \cdot (\text{in_channels} \cdot k \cdot k + 1)$$

- `Conv2d(1, 16, 3)`: $16 \cdot (1 \cdot 9 + 1) = 160$
- `Conv2d(16, 32, 3)`: $32 \cdot (16 \cdot 9 + 1) = 4\,640$

Parámetros de una capa convolucional

Cada filtro tiene forma `in_channels`$\times k \times k$, más un sesgo:

$$\text{parámetros} = \text{out_channels} \cdot (\text{in_channels} \cdot k \cdot k + 1)$$

- `Conv2d(1, 16, 3)`: $16 \cdot (1 \cdot 9 + 1) = 160$
- `Conv2d(16, 32, 3)`: $32 \cdot (16 \cdot 9 + 1) = 4\,640$

Observación

El número de parámetros **no depende** del tamaño de la imagen: el mismo filtro se reutiliza en toda la imagen.

Tamaño de salida del pooling

Misma fórmula, pero con $p = 0$ y, por defecto, $s = k$:

$$n_{sal} = \left\lfloor \frac{n-k}{k} \right\rfloor + 1 = \left\lfloor \frac{n}{k} \right\rfloor$$

- `MaxPool2d(2)`: $28 \to 14$, $14 \to 7$, $7 \to 3$ (el residuo se descarta).
- El pooling **no cambia el número de canales**.
- No tiene parámetros que aprender.

Tamaño de salida del pooling

Misma fórmula, pero con $p = 0$ y, por defecto, $s = k$:

$$n_{sal} = \left\lfloor \frac{n-k}{k} \right\rfloor + 1 = \left\lfloor \frac{n}{k} \right\rfloor$$

- `MaxPool2d(2)`: $28 \to 14$, $14 \to 7$, $7 \to 3$ (el residuo se descarta).
- El pooling **no cambia el número de canales**.
- No tiene parámetros que aprender.

Idea práctica

Con imágenes de tamaño par, cada `MaxPool2d(2)` divide alto y ancho entre 2.

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)`
2. `MaxPool2d(2)`
3. `Conv2d(16, 32, kernel_size=3)`
4. `MaxPool2d(2)`
5. `Flatten`

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)` $\rightarrow 16 \times 28 \times 28$
2. `MaxPool2d(2)`
3. `Conv2d(16, 32, kernel_size=3)`
4. `MaxPool2d(2)`
5. `Flatten`

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)` $\rightarrow 16 \times 28 \times 28$
2. `MaxPool2d(2)` $\rightarrow 16 \times 14 \times 14$
3. `Conv2d(16, 32, kernel_size=3)`
4. `MaxPool2d(2)`
5. `Flatten`

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)` $\rightarrow 16 \times 28 \times 28$
2. `MaxPool2d(2)` $\rightarrow 16 \times 14 \times 14$
3. `Conv2d(16, 32, kernel_size=3)` $\rightarrow 32 \times 12 \times 12$
4. `MaxPool2d(2)`
5. `Flatten`

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)` $\rightarrow 16 \times 28 \times 28$
2. `MaxPool2d(2)` $\rightarrow 16 \times 14 \times 14$
3. `Conv2d(16, 32, kernel_size=3)` $\rightarrow 32 \times 12 \times 12$
4. `MaxPool2d(2)` $\rightarrow 32 \times 6 \times 6$
5. `Flatten`

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)` $\rightarrow 16 \times 28 \times 28$
2. `MaxPool2d(2)` $\rightarrow 16 \times 14 \times 14$
3. `Conv2d(16, 32, kernel_size=3)` $\rightarrow 32 \times 12 \times 12$
4. `MaxPool2d(2)` $\rightarrow 32 \times 6 \times 6$
5. `Flatten` $\rightarrow 32 \cdot 6 \cdot 6 = 1152$

Práctica: calcula las dimensiones

Entrada: imagen CIFAR-10, $3 \times 32 \times 32$.

1. `Conv2d(3, 16, kernel_size=5)` $\rightarrow 16 \times 28 \times 28$
2. `MaxPool2d(2)` $\rightarrow 16 \times 14 \times 14$
3. `Conv2d(16, 32, kernel_size=3)` $\rightarrow 32 \times 12 \times 12$
4. `MaxPool2d(2)` $\rightarrow 32 \times 6 \times 6$
5. `Flatten` $\rightarrow 32 \cdot 6 \cdot 6 = 1152$

Entonces la primera capa lineal debe ser `Linear(1152, ...)`.

Truco: deja que PyTorch haga la cuenta

Pasa una imagen falsa por la parte convolucional y observa la forma:

```
import torch, torch.nn as nn

parte_conv = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=5), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(16, 32, kernel_size=3), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten()
)

x = torch.zeros(1, 3, 32, 32)       # (batch, canales, alto, ancho)
print(parte_conv(x).shape)          # torch.Size([1, 1152])
```

El segundo valor es el número de entradas de la primera `Linear`.

Tres capas nuevas, el mismo Sequential

Una CNN se arma igual que la red de *moons*: apilando capas.

```
nn.Conv2d(in_channels, out_channels, kernel_size, padding)
nn.MaxPool2d(kernel_size)
nn.Flatten()
```

- `Conv2d`: aplica `out_channels` filtros; `in_channels` es 1 en escala de grises.
- `MaxPool2d(2)`: reduce alto y ancho a la mitad.
- `Flatten`: convierte los mapas de características en un vector para las capas `Linear`.

Proyecto final: detectar neumonía en radiografías



- **PneumoniaMNIST**: radiografías de tórax en escala de grises, 28×28.
- Clasificación binaria: normal vs. neumonía.
- Disponible con la biblioteca `medmnist` (`pip install medmnist`).

Paso 1: Datos

```
from medmnist import PneumoniaMNIST
from torchvision import transforms
from torch.utils.data import DataLoader

tf = transforms.ToTensor()   # convierte a tensor y escala a [0,1]
train_ds = PneumoniaMNIST(split='train', download=True, transform=tf)
test_ds  = PneumoniaMNIST(split='test',  download=True, transform=tf)

train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)
test_dl  = DataLoader(test_ds,  batch_size=64)
```

Paso 2: Modelo (CNN pequeña con Sequential)




```
modelo = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),   # 1x28x28   -> 16x28x28
    nn.ReLU(),
    nn.MaxPool2d(2),                              #           -> 16x14x14
    nn.Conv2d(16, 32, kernel_size=3, padding=1),  #           -> 32x14x14
    nn.ReLU(),
    nn.MaxPool2d(2),                              #           -> 32x7x7
    nn.Flatten(),                                 #           -> 1568
    nn.Linear(32 * 7 * 7, 64),
    nn.ReLU(),
    nn.Linear(64, 2)                              # 2 clases
)
```

Dimensiones y parámetros de nuestra CNN

Capa	**Forma de salida**	**Parámetros**
Entrada	$1 \times 28 \times 28$	0
Conv(1→16, k=3, p=1) + ReLU	$16 \times 28 \times 28$	160
MaxPool(2)	$16 \times 14 \times 14$	0
Conv(16→32, k=3, p=1) + ReLU	$32 \times 14 \times 14$	4 640
MaxPool(2)	$32 \times 7 \times 7$	0
Flatten	1568	0
Linear(1568→64) + ReLU	64	100 416
Linear(64→2)	2	130
Total		**105 346**

Casi todos los parámetros están en la primera capa `Linear`.

Pasos 3, 4 y 5: Pérdida, optimizador y entrenamiento




```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
modelo = modelo.to(device)
criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo.parameters(), lr=0.001)

for epoca in range(10):
    for imgs, etiquetas in train_dl:
        imgs = imgs.to(device)
        etiquetas = etiquetas.squeeze().long().to(device)  # (N,1) -> (N)

        optimizador.zero_grad()
        loss = criterio(modelo(imgs), etiquetas)
        loss.backward()
        optimizador.step()
    print(f"Epoca {epoca+1} | Perdida: {loss.item():.4f}")
```

Evaluación en el conjunto de prueba

```
modelo.eval()
correctas, total = 0, 0
with torch.no_grad():
    for imgs, etiquetas in test_dl:
        imgs = imgs.to(device)
        etiquetas = etiquetas.squeeze().long().to(device)
        pred = modelo(imgs).argmax(dim=1)
        correctas += (pred == etiquetas).sum().item()
        total += etiquetas.size(0)
print(f"Precision en prueba: {correctas/total:.2%}")
```

Actividad 2 (proyecto)

1. Entrena la CNN y reporta la precisión en prueba.
2. Prueba cambiar el número de filtros o añadir una tercera capa convolucional.
3. Compara contra una red densa (`Flatten` + `Linear`) con la misma metodología.
4. Reflexión: en medicina, ¿basta con la precisión? ¿Qué errores son más graves?

¿Dónde estamos?

1. Introducción al aprendizaje automático
2. Redes neuronales y backpropagation
3. Construyendo redes con PyTorch
   - Herramientas básicas
   - Descenso de gradiente paso a paso
   - Nuestra primera red: moons
4. Introducción a CNN con PyTorch
   - Calculando dimensiones
   - Las capas de una CNN con Sequential
   - Proyecto: PneumoniaMNIST
5. Cierre

Lo que aprendimos: la metodología completa



1. **Datos**: tensores (puntos 2D o imágenes).
2. **Modelo**: `nn.Sequential`.
3. **Pérdida**: `CrossEntropyLoss`.
4. **Optimizador**: `Adam` o `SGD`.
5. **Bucle**: `zero_grad` → `forward` → `loss` → `backward` → `step`.

Cambiar de *moons* a radiografías solo modificó los pasos 1 y 2.

Para seguir aprendiendo

- Documentación y tutoriales: `https://pytorch.org/tutorials`
- MedMNIST: `https://medmnist.com`
- Siguientes temas: regularización (*dropout*, *weight decay*), aumento de datos, transfer learning, redes recurrentes y *transformers*.

¡Gracias!